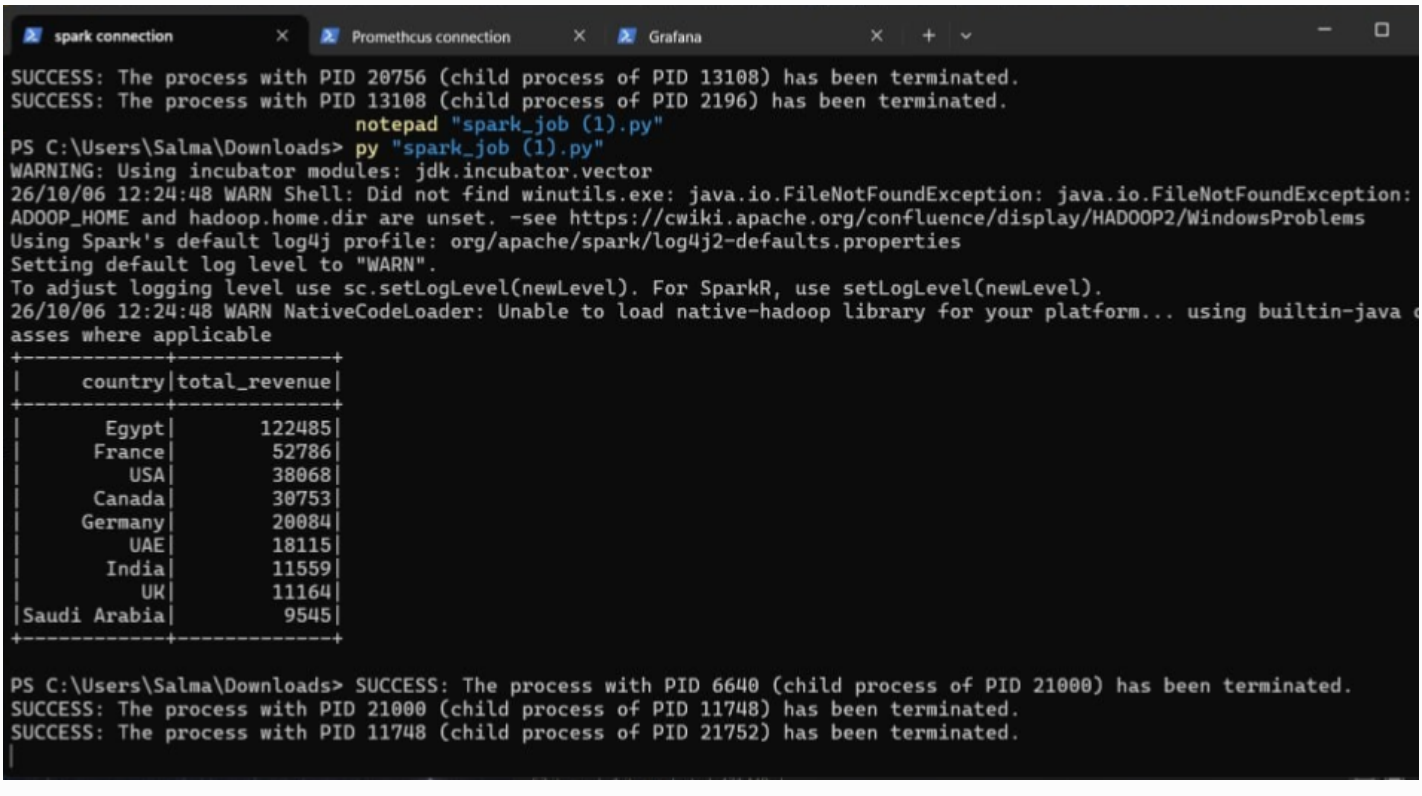
Downloading customers.csv , orders.csv from GrafanaTask folder

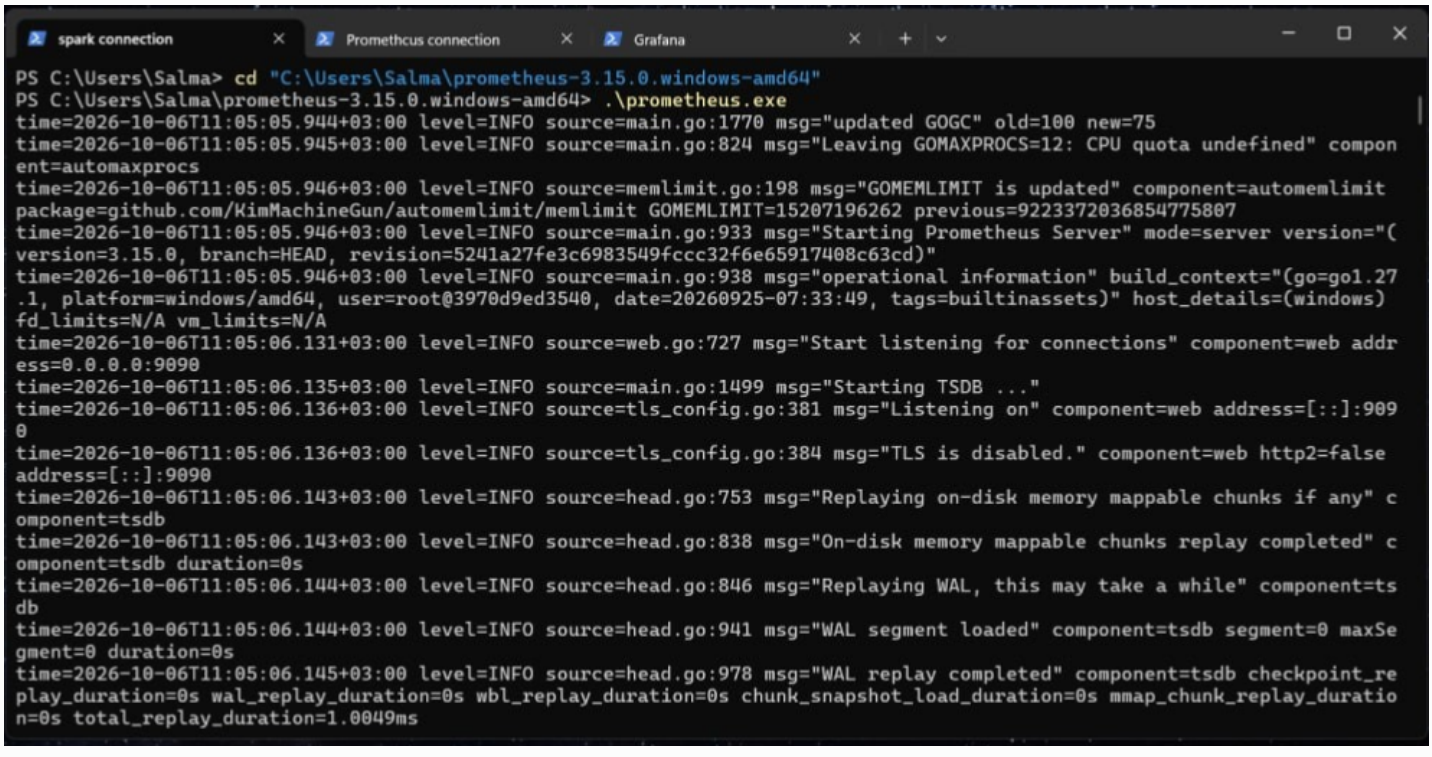
Opening notepad named spark_job.py to perform filtering and joining filter data , enabling promethcusmetrics

Code is uploaded in a file named spark_job

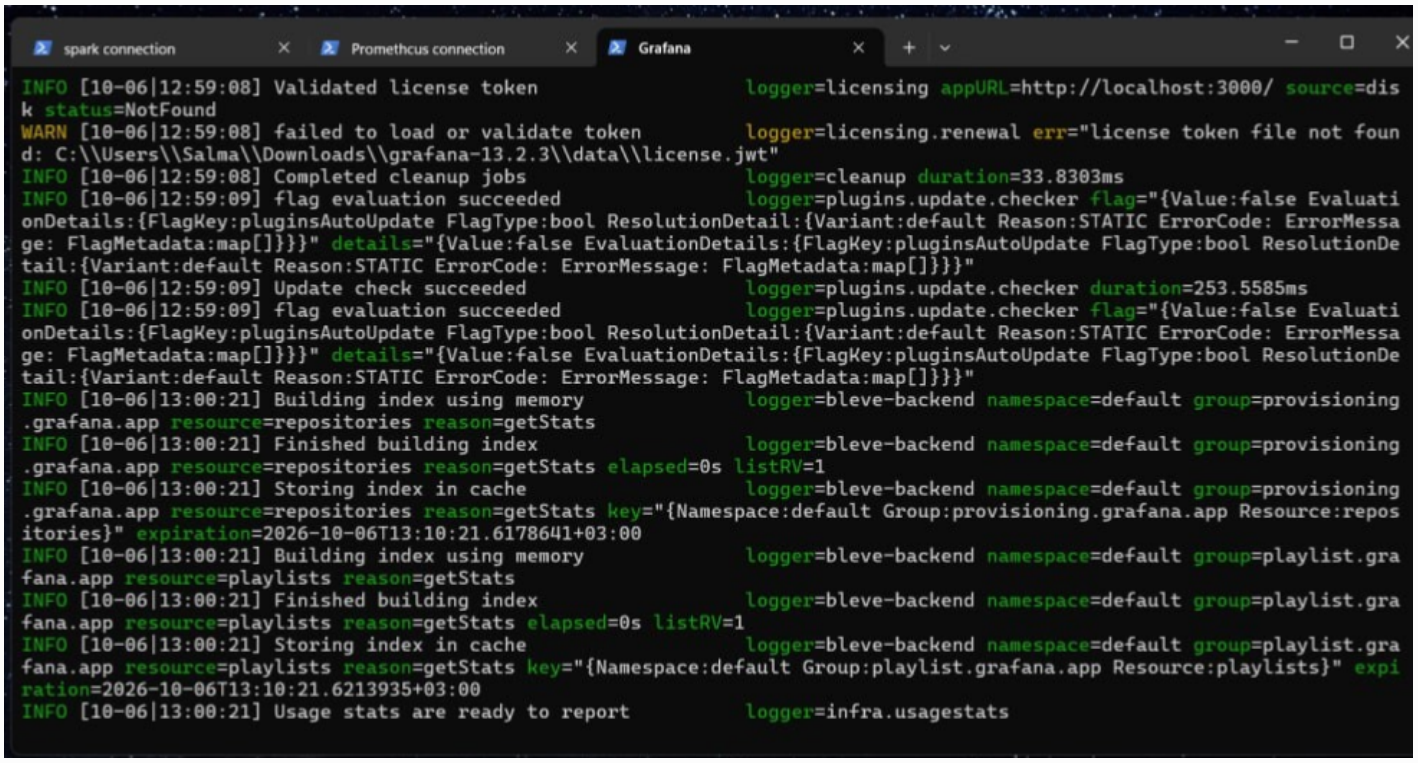
Run the file in powershell terminal to start spark connection



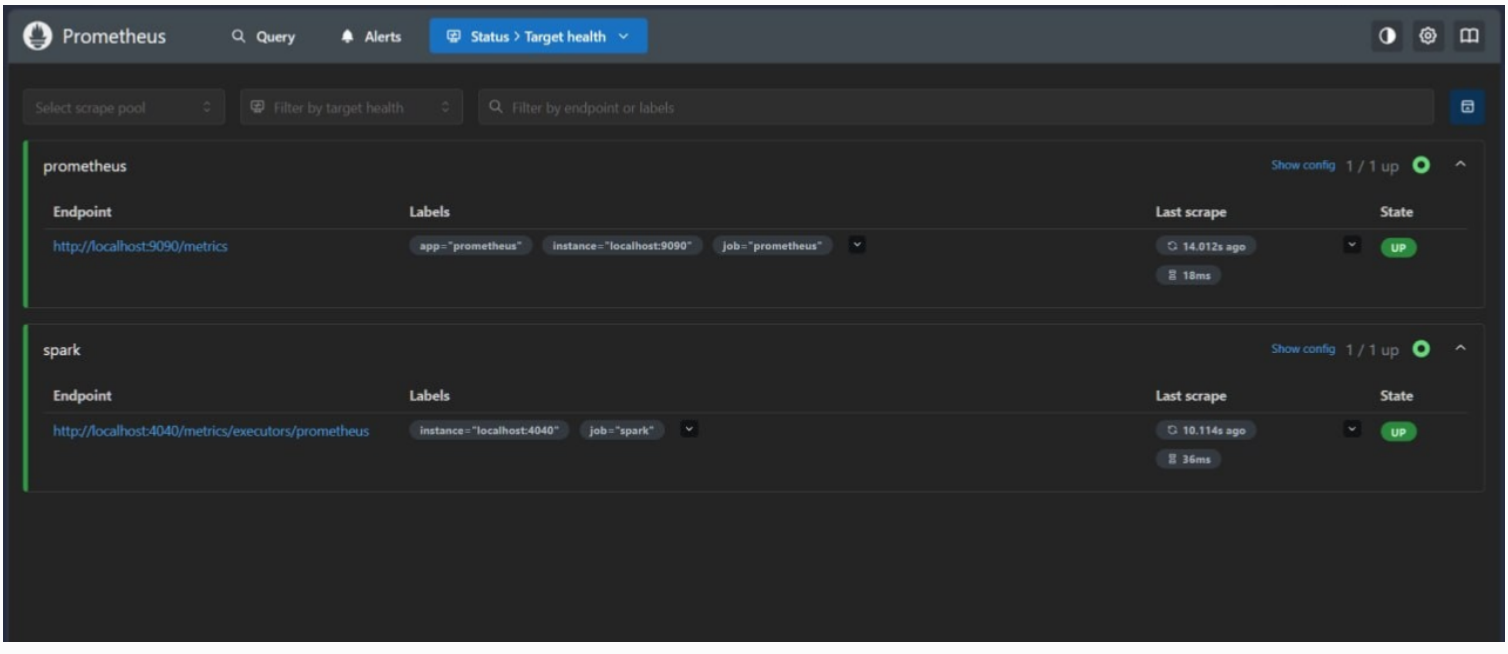
Downloading Promethcus , run , server connection



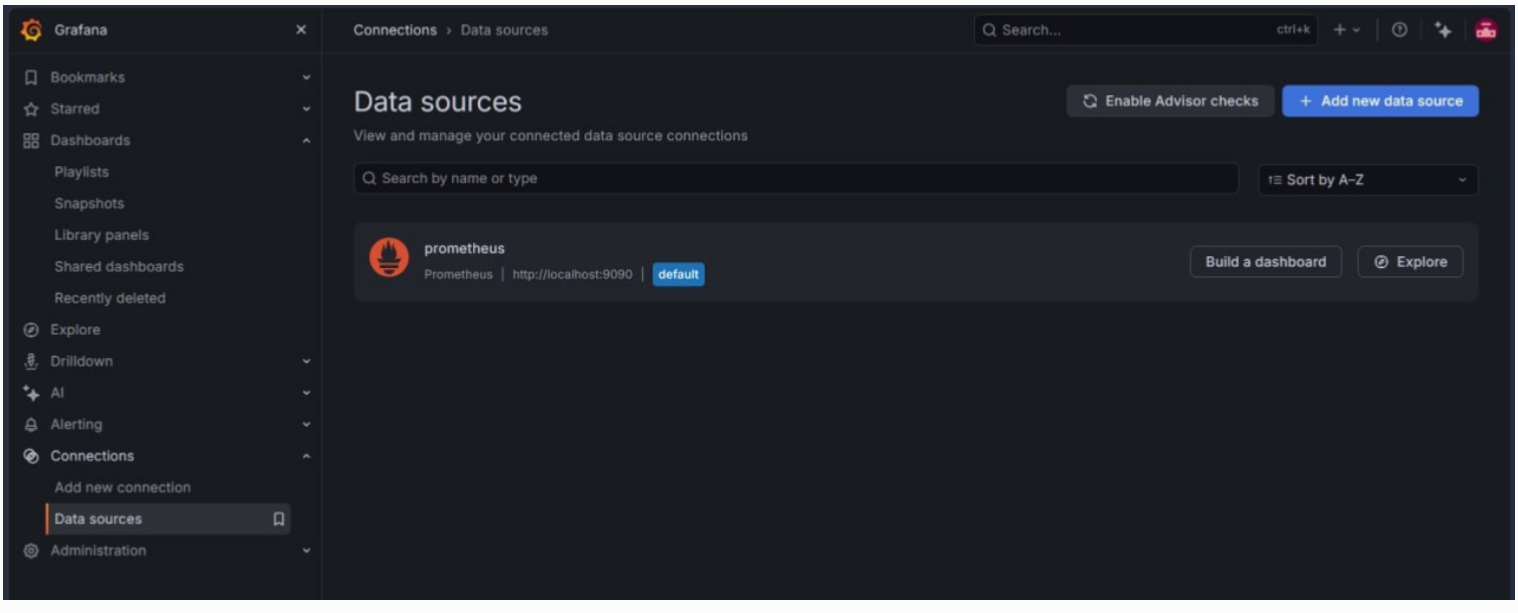
Downloading Grafana , run , server connection

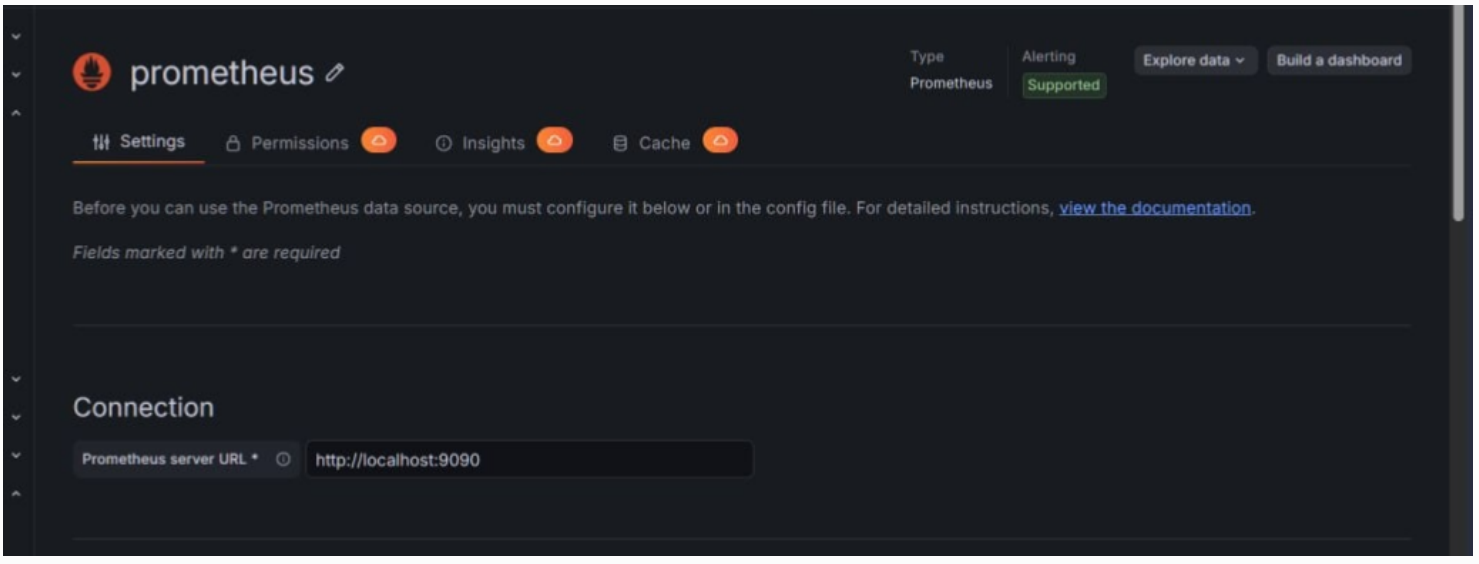


Active Promethcus Scrape targets

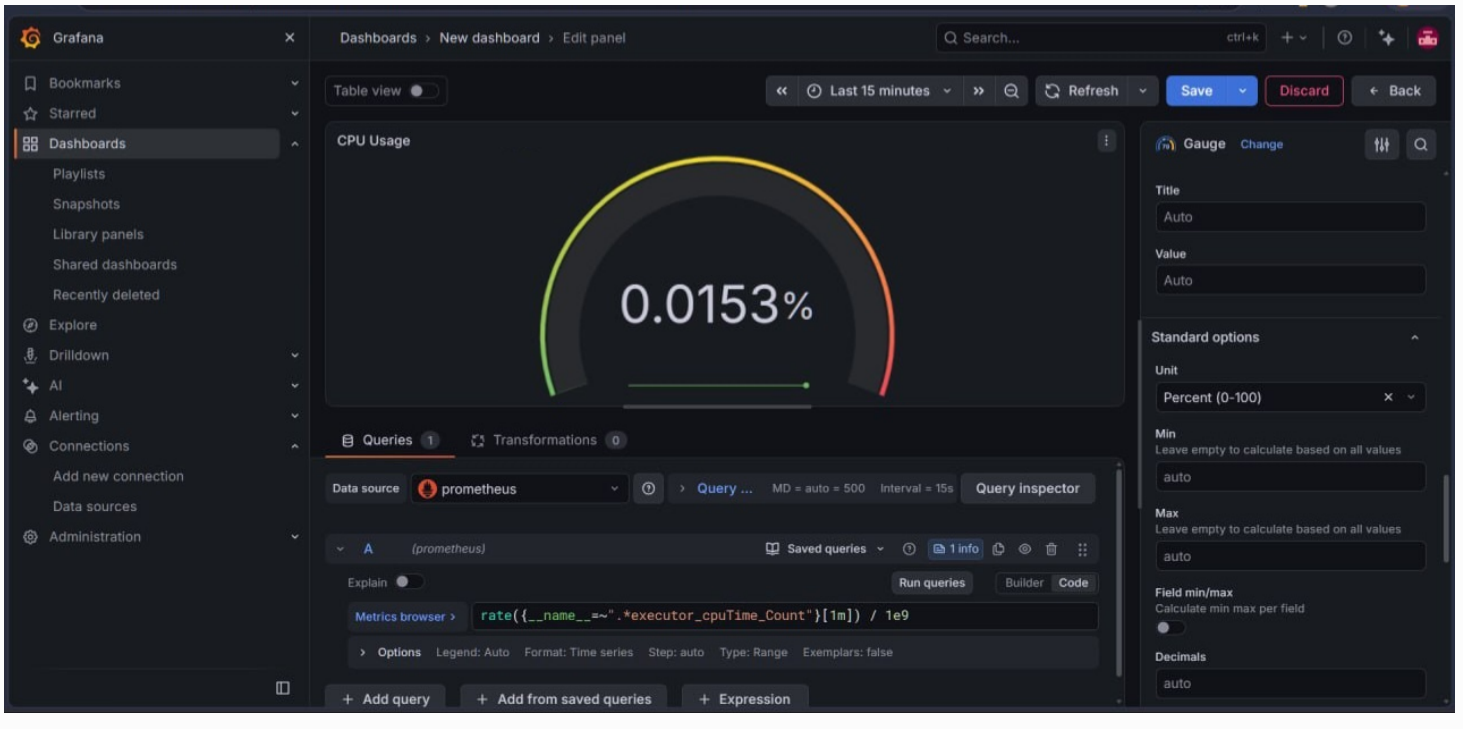


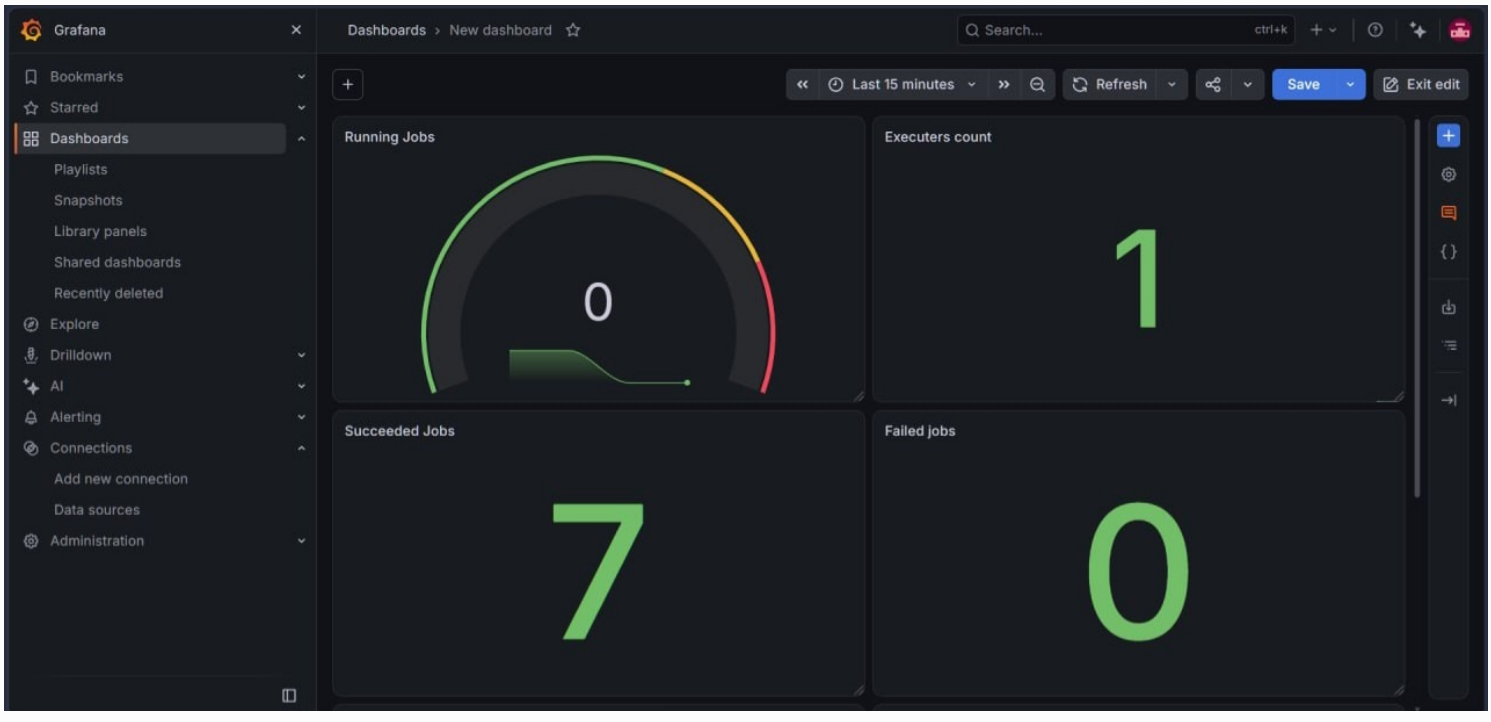
Grafana connection to Promethcus

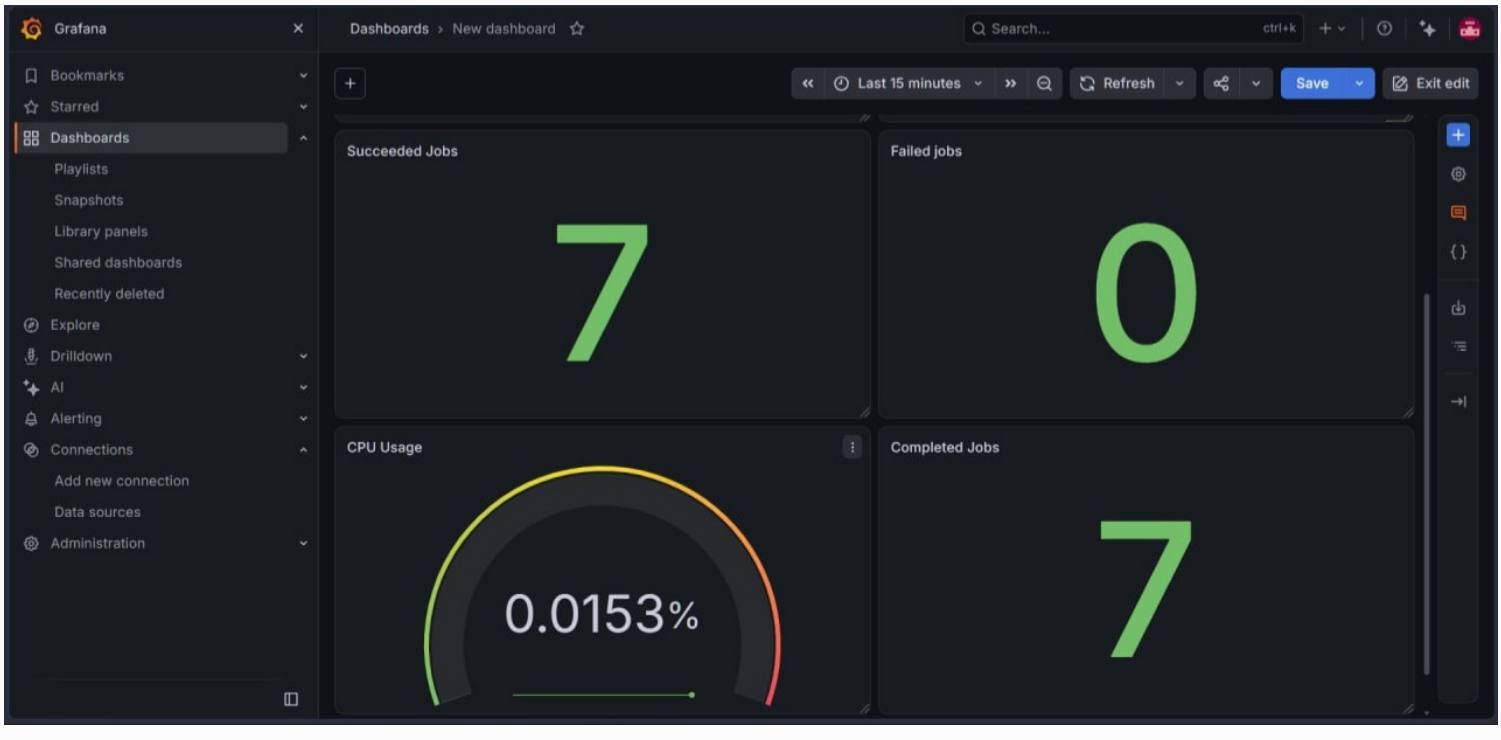




Clicking on Dashboards , adding new dashboard and applying queries to panels as one query per panel switching from builder to code , selecting suitable visuals and units




Grafana
Dashboards › New dashboard
Search...
ctrl+k
Bookmarks
Starred
Dashboards
Playlists
Snapshots
Library panels
Shared dashboards
Recently deleted
Explore
Drilldown
AI
Alerting
Connections
Add new connection
Data sources
Administration
Last 15 minutes
Refresh
Save
Exit edit
Running Jobs
0
Executers count
1
Succeeded Jobs
7
Failed jobs
0


Grafana
Dashboards › New dashboard
Search...
ctrl+k
Bookmarks
Starred
Dashboards
Playlists
Snapshots
Library panels
Shared dashboards
Recently deleted
Explore
Drilldown
AI
Alerting
Connections
Add new connection
Data sources
Administration
Last 15 minutes
Refresh
Save
Exit edit
Succeeded Jobs
7
Failed jobs
0
CPU Usage
0.0153%
Completed Jobs
7

1. Running Jobs

PromQL Query:

{__name__=~".*DAGScheduler_job_activeJobs_Value"}

Grafana Unit:short

2. Completed Jobs

PromQL Query:

{__name__=~".*DAGScheduler_job_allJobs_Value"}

Grafana Unit:short

3. Failed Jobs

PromQL Query:

{__name__=~".*DAGScheduler_job_failedJobs_Value"} or vector(0)

Grafana Unit:short

4. CPU Usage

PromQL Query:

```
rate({__name__=~".*executor_cpuTime_Count"}[1m]) / 1e9
```

Grafana Unit:Percent (0.0–1.0)

5. Executors Count

PromQL Query:

```
count({__name__=~".*executor_threadpool_activeTasks_Value"})
```

Grafana Unit:short